Explainability in Reinforcement Learning

Maxime Alaarabiou

Wednesday 2nd September, 2026

Compiled on 2/9/2026 at 14:28

Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), the training paradigm used to optimize deep Neural Network (NN). The objective is to provide the necessary foundation in DL for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial NN.

Section 1.4: An explanation of how the learning process shapes NN.

Section 1.5: A discussion of how such models are evaluated.

Section 1.6: A summary of the concepts introduced in the chapter and a discussion of the theoretical limitations of DL.

1.1 Historical Developments

NN and DL emerged from an ambition to study the expressive capabilities of organic brain. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [McCulloch and Pitts, 1943]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications.

A major step toward operationalizing the idea of NNs was achieved with the perceptron algorithm in 1958 [Rosenblatt, 1958]. It was designed to adjust its parameters from input–output pairs in order to approximate a target function. The perceptron fundamental limitations were exposed in 1969, showing that it could not learn functions as simple as the exclusive-or [Minsky and Papert, 1969]. At the same time, it was demonstrated that stacking multiple perceptrons could, in principle, represent such functions. However, no effective training procedure

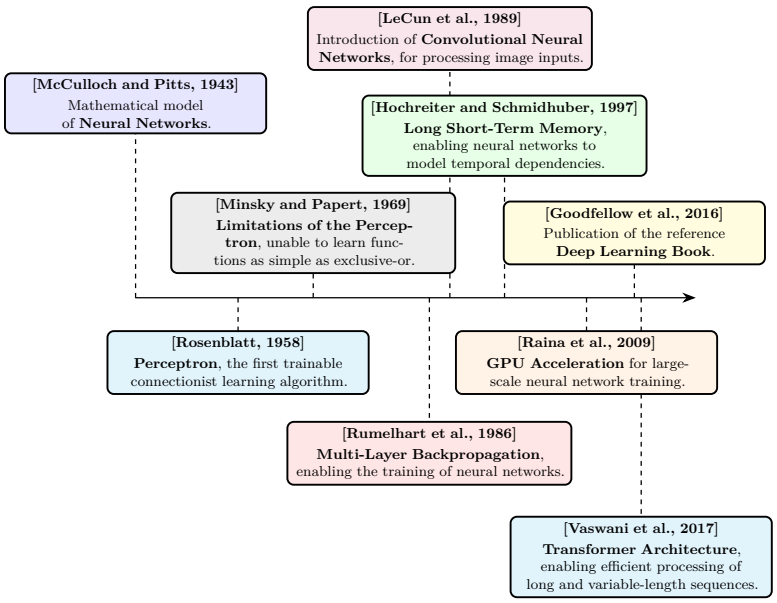


Figure 1.1: Evolution of deep learning.

for these multi-layer architectures was available at the time. It was only in 1986 that researchers introduced gradient descent as an effective method for training deep NN [Rumelhart et al., 1986]. From this point onward, the emergence of DL is retrospectively situated, even though the term itself appeared later. DL is defined as the training of deep NN using gradient-based optimization methods.

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [Hornik et al., 1989], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [Hilbert and López, 2011]. The use of graphics processing units for large-scale training, demonstrated in 2009, greatly increased computational capacity and made it feasible to train large-scale models [Raina et al., 2009, Krizhevsky et al., 2012].

At the same time, researchers developed architectures specifically designed for different types of data, allowing DL to address an increasingly wide range of problems. Convolutional Neural Network (CNN) were introduced in 1989 to process images, using local, weight-shared filters that exploit the spatial structure of pixel grids [LeCun et al., 1989]. Long Short-Term Memory (LSTM) networks were later proposed in 1997 to process temporal sequences, introducing

gating mechanisms able to retain relevant information over long time horizons [Hochreiter and Schmidhuber, 1997]. More recently, the Transformer architecture, relying entirely on the attention mechanism, was introduced in 2017 to process long, variable-length sequences of strongly interconnected elements, allowing every element to attend directly to every other element regardless of their distance within the sequence [Vaswani et al., 2017]. This growing body of knowledge was consolidated in 2016 in a reference textbook that formalized DL as a coherent field of study [Goodfellow et al., 2016].

These changes paved the way for the rise of DL as it is known today. DL has achieved remarkable success across a wide range of domains, including computer vision [Krizhevsky et al., 2012], complex games [Silver et al., 2017], content recommendation [Covington et al., 2016], bio-chemistry [Jumper et al., 2021], as well as the now essential digital assistants based on Large Language Model (LLM) [Brown et al., 2020]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

Figure 1.1 provides a timeline of the key milestones that shaped deep learning.

1.2 Objective

Now that the main developments that led to the rise of DL have been outlined, it is necessary to formalize what is actually being achieved when performing DL. To ensure clarity, the supervised learning setting will be considered, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. Since a NN is a parametric function, its parameters are progressively updated during training so as to make it a better approximation of the target function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive parameter updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."
> [Mitchell, 1997]

When this formulation is adapted to DL, the following correspondences hold:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.
- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

The function to be approximated is denoted by f. As with any function, it defines a mapping from an input to an output: $x \in \mathcal{X}$ denotes an input and $y \in \mathcal{Y}$ the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition, both $\mathcal{X}$ and $\mathcal{Y}$ are assumed to be vector spaces throughout this manuscript. This assumption is adopted solely to facilitate the presentation, as the framework can be readily extended to more general settings. The notation $\hat{\cdot}$ indicates that a given object is an approximation of another. Since the goal of the trained NN is to approximate f, this approximation is denoted by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, only a finite set of observations generated by f is available, organized in what is called a dataset. To distinguish between the different samples, an index i is introduced, allowing input–output pairs of the form $(x^{(i)}, y^{(i)})$ to be defined. This leads to the following formulation for a dataset $\mathcal{D}$ of size N, where N denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}. \tag{1.1}$$

Now that the representation of the target function f has been established, the next question is what makes a function $\hat{f}$ a good approximation of it. For a given input $x^{(i)}$, this means that the NN output $\hat{y}^{(i)}$ should be close to the true output $y^{(i)}$. To quantify the quality of this approximation, a function $\mathcal{L}$ is introduced that measures the discrepancy between predictions and target values. This function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the Mean Squared Error (MSE), defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2. \tag{1.2}$$

Thanks to this loss, it is now possible to rigorously define the performance measure P. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}). \tag{1.3}$$

This formulation captures the mathematical core of the learning objective, namely finding a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$.

Since the target function f may take many different forms, $\hat{f}$ must be expressive enough to approximate a broad range of such functions. NN architectures are well suited to this requirement, as they can represent a wide range of functional mappings, whose composition is detailed in the following section.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation, the focus is placed on the MultiLayer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of DL. All the essential mechanisms are present in it, making it a particularly suitable framework for introducing key concepts.

This section therefore begins by examining the artificial neuron, the fundamental computational unit underlying the MLP. It then explains how these neurons are organized into layers. Finally, it concludes with a general description of the NN as a whole.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons, where these inputs are represented as real-valued numbers. It produces an output, also a real-valued number, which is transmitted to subsequent neural units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \ldots, a_n)$, its activation a is given by:

$$a = \sigma\left(\sum_{i=1}^{n} w_i a_i + b\right), \tag{1.4}$$

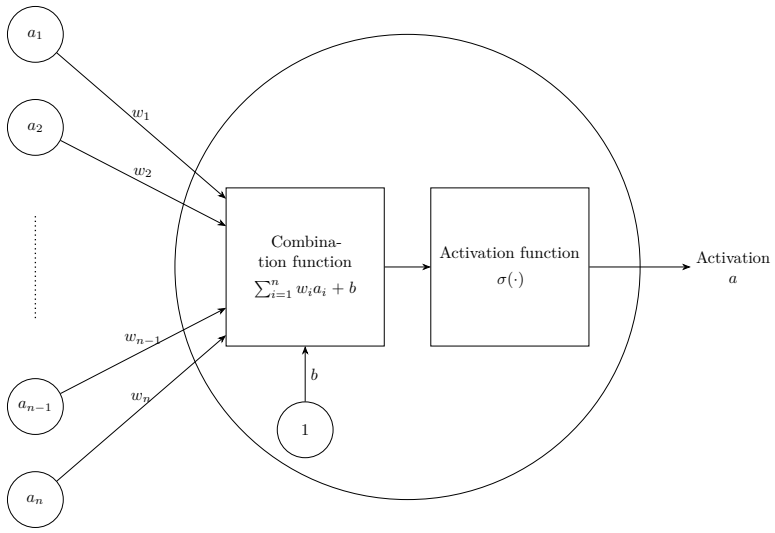


Figure 1.2: Artificiale neurone.

where:

- w_i is the weight associated with the i-th incoming connection,
- b is the bias,
- σ is the activation function.

Figure 1.2 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w, relative to the others, indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer

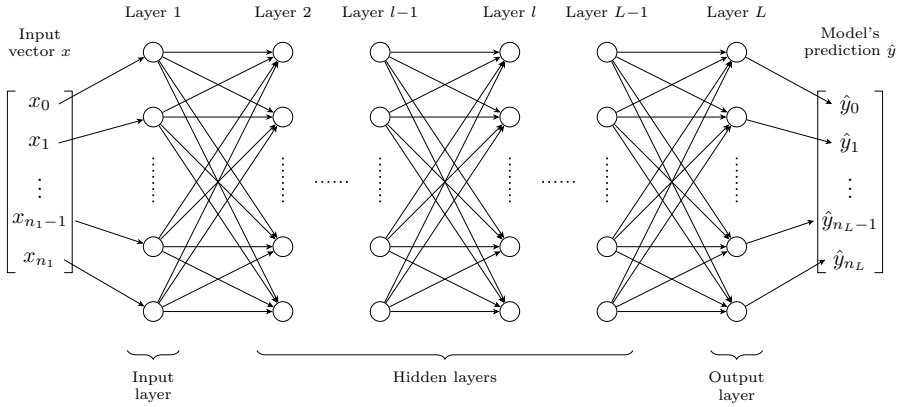


Figure 1.3: Multilayer Perceptron Architecture.

and sends its output to all neurons in the following layer. As a result, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers (also referred to as intermediate layers), which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer, as its neurons directly encode the components of the input vector x. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [Krizhevsky et al., 2012]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \ldots, a_{n_l}^{(l)}]. \tag{1.5}$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \tag{1.6}$$

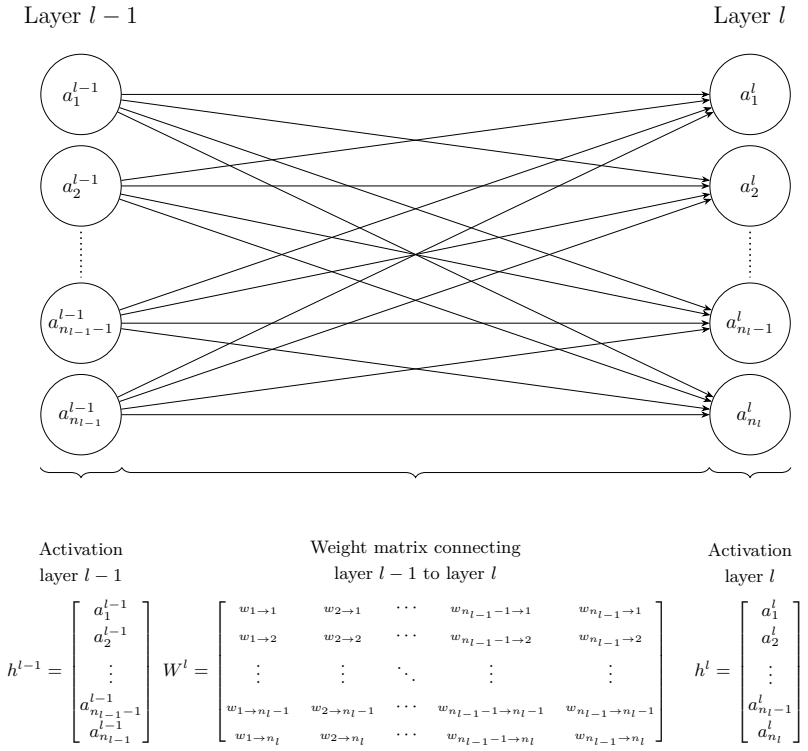


Figure 1.4: Interaction between layers.

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer l,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l.

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions, and in practice, modern architectures often include many more. Theoretical results have shown that, for a fixed number of parameters, increasing the number of layers enhances the expressive capacity of NN [Telgarsky, 2016]. Each layer can be viewed as an additional stage in the computation performed by the network. This suggests that deeper networks can represent increasingly complex functions by composing a larger number of transformations. The precise relationship between depth and function complexity remains primarily supported by empirical observations rather than formal theoretical justification [Goodfellow et al., 2016].

1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L)}\right) = \hat{y}. \tag{1.7}$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) & \text{if } 2 \le l \le L. \end{cases} \tag{1.8}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}. \tag{1.9}$$

In this manuscript, θ is a vector in $\mathbb{R}^P$, where P denotes the total number of parameters, such that $\theta = [w_1, \dots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, can enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When referring to the architecture of $\hat{f}$, the number of layers, the number of neurons per layer, and the choice of activation functions are considered. The parameters correspond to θ, which controls the strength of the connections between neurons. The following sections examine how these parameters can be adjusted to obtain a better approximation of the target function.

1.4 Learning

Now that the structure of a NN has been detailed, the next step is to examine how its parameters are adjusted to accomplish the assigned task. This brings the discussion to the core of DL, namely the learning process itself. As a reminder, the objective of training a NN is to adjust its parameters so that the network provides a good approximation of the target function. Mathematically, this objective is expressed as:

$$\min_\theta \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.10}$$

As shown in Equation 1.10, the loss for example i depends on the input-target pair $(x^{(i)}, y^{(i)})$, the loss function $\mathcal{L}$, the network architecture $\hat{f}$, and the parameters θ. Since the dataset, loss function, and network architecture are fixed

during training, only the parameters θ are optimized. To simplify the notation, only the dependence on θ is kept, and the loss for example i is written as

$$\mathcal{L}^{(i)}(\theta) = \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.11}$$

Now that the learning objective has been defined, the next question is how to modify the parameters θ to minimize the loss. The NN architecture $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, the optimum cannot be determined analytically by directly studying the properties of the function.

Instead, approximate optimization methods are required to find suitable parameter values for NN. The class of algorithms used for this optimization is known as gradient descent methods. Numerous variants of gradient descent have been developed for NN training. For simplicity, the focus here is on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

This section presents the learning process in three steps. The first derives the gradient of the loss using the chain rule. The second shows how this gradient is used to update the network parameters. The third discusses the convergence properties of the resulting algorithm.

1.4.1 Computing the Gradient

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. This initial state is denoted by $\theta^{(1)}$. The goal is to iteratively update this vector so that the network gets better at approximating the target function. To do so, it is necessary to quantify the impact of each parameter w_k on the loss $\mathcal{L}^{(i)}(\theta)$. This is done using the partial derivative of the loss with respect to each parameter, denoted by $\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k}$.

Computing this derivative is not straightforward because a NN is composed of a sequence of functions, and the parameter w_k can belong to any layer of the network. Its influence on the loss must therefore be propagated through all subsequent layers. To describe these dependencies, each layer can be viewed as a function, denoted by $\phi^{(l)}$ for layer l. This function takes the activation vector $\mathbf{h}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ produced by the previous layer as input. It produces the activation vector $\mathbf{h}^{(l)} \in \mathbb{R}^{n_l}$ of the current layer. This can be expressed as

$$\phi^{(l)} : \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}, \qquad \mathbf{h}^{(l)} = \phi^{(l)}(\mathbf{h}^{(l-1)}). \tag{1.12}$$

Each layer function can have multiple inputs and multiple outputs, so the ordinary derivative is not sufficient to describe how its outputs change with respect to its inputs. The Jacobian matrix extends the derivative to functions with multiple inputs and outputs, quantifying how each output component changes with respect to each input component. Consider the function $\phi^{(l)}$ and its Jacobian $J_{\phi^{(l)}}$ evaluated at $\mathbf{h}^{(l-1)}$. The Jacobian collects the partial derivatives

of each output component with respect to each input component:

$$J_{\phi^{(l)}}(\mathbf{h}^{(l-1)}) = \begin{bmatrix} \frac{\partial h_1^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_1^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{n_l}^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_{n_l}^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \end{bmatrix}. \tag{1.13}$$

Since a NN is composed of several layers, a change in an early layer can affect the outputs of all subsequent layers. It is therefore necessary to propagate this change through the sequence of layer functions until it reaches the network output. The chain rule provides the mathematical tool required to describe this propagation. To illustrate this, consider two consecutive layers, $\phi^{(l)}$ and $\phi^{(l+1)}$. The chain rule states that the Jacobian of their composition is given by the product of their Jacobians. Each Jacobian is evaluated at the input received by the corresponding layer:

$$J_{\phi^{(l+1)} \circ \phi^{(l)}}\left(\mathbf{h}^{(l-1)}\right) = J_{\phi^{(l+1)}}\left(\mathbf{h}^{(l)}\right) J_{\phi^{(l)}}\left(\mathbf{h}^{(l-1)}\right). \tag{1.14}$$

The network is composed of a sequence of layer functions $\phi^{(1)}, \ldots, \phi^{(L)}$. Each layer depends only on the output of the previous layer. Consider a parameter w_k belonging to layer l. A change in w_k first affects the output $\mathbf{h}^{(l)}$ of its own layer. This change then propagates through the subsequent layers, affecting $\mathbf{h}^{(l+1)}$, and so on. The effect reaches the output $\mathbf{h}^{(L)}$, from which the loss $\mathcal{L}^{(i)}(\theta)$ is computed. This layer-by-layer propagation is referred to as backpropagation:

$$\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k} = J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})\, J_{\phi^{(L)}}(\mathbf{h}^{(L-1)}) \cdots J_{\phi^{(l+1)}}(\mathbf{h}^{(l)})\, J_{\phi^{(l)}}(w_k), \tag{1.15}$$

where:

- $J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})$ denotes the Jacobian of $\mathcal{L}$ with respect to its first argument, evaluated at $(\mathbf{h}^{(L)}, y^{(i)})$,
- each subsequent $J_{\phi^{(j+1)}}(\mathbf{h}^{(j)})$ denotes the Jacobian of the function $\phi^{(j+1)}$ with respect to its input $\mathbf{h}^{(j)}$,
- $J_{\phi^{(l)}}(w_k)$ denotes the Jacobian of the function $\phi^{(l)}$ with respect to the parameter w_k.

Once $\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k}$ can be computed for every parameter, all of these values are stacked into one vector, called the gradient:

$$\nabla_\theta \mathcal{L}^{(i)}(\theta) = \left[\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_P}\right]. \tag{1.16}$$

Each component answers the same question for a different parameter, indicating whether a small increase in that parameter increases or decreases the loss, and by how much. A positive value means increasing the parameter increases the loss, whereas a negative value means the opposite. The gradient thus indicates, for every parameter, which direction to move in order to reduce the loss.

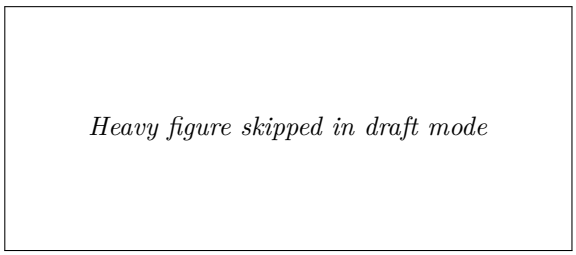


Figure 1.5: *Figure skipped in draft mode (set* `\draftfiguresfalse` *to render it).*

1.4.2 Update rule

Now that information is available on how changes in the parameters affect the loss, the objective is to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, the gradient is followed in the opposite direction because it indicates the direction in which the loss increases, whereas the objective is to reduce it. This leads to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_\theta \mathcal{L}^{(i)} \left(\theta^{(n)}\right), \tag{1.17}$$

where $\eta > 0$ is the learning rate, which controls the step size, and $\nabla_\theta \mathcal{L}^{(i)} \left(\theta^{(n)}\right)$ denotes the gradient of the loss with respect to the parameters, evaluated at the current values $\theta^{(n)}$.

The parameter η must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta^{(1)}, \theta^{(2)}, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

Algorithm 1 summarizes this whole training procedure.

To develop an intuition for this update rule, Figure 1.5 illustrates it applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum. The point $\theta^{(1)}$ represents the initial parameter values, with w_1 and w_2 randomly initialized, and the subsequent points, from $\theta^{(2)}$ to $\theta^{(6)}$, represent the successive updates produced by applying Equation 1.17, progressively moving the trajectory toward the minimum of the loss function.

1.4.3 Convergence Properties

Having established how SGD updates the parameters, a natural question is whether this procedure is guaranteed to converge. It has been mathematically shown that gradient descent applied to a convex loss with respect to the parameters converges to a global optimum, independently of the initialization. However, NN training involves highly non-convex loss surfaces, so the optimization trajectory becomes strongly dependent on the initial conditions, and no such guarantee holds. Figure 1.6 illustrates the effect of non-convex functions, different starting points lead to distinct local minima.

Algorithm 1: Stochastic Gradient Descent

Input:
Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$
Architecture $\hat{f}$
Loss function $\mathcal{L}$
Learning rate $\eta > 0$
Initial parameters θ (optional)
Output:
Trained parameters θ

---

if θ *is given* **then**
  $\theta^{(1)} \leftarrow \theta$;
else
  Initialize $\theta^{(1)}$ randomly;
$n \leftarrow 1$;
while *stopping criterion not satisfied* **do**
  Select a random example $(x^{(i)}, y^{(i)}) \in \mathcal{D}$;
  Model prediction for the selected example
  $\hat{y}^{(i)} \leftarrow \hat{f}_{\theta^{(n)}}(x^{(i)})$;
  Per-example loss between the prediction and the ground truth
  $\mathcal{L}^{(i)}\left(\theta^{(n)}\right) \leftarrow \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$;
  Gradient of the per-example loss, since it is differentiable
  $g \leftarrow \nabla_\theta \mathcal{L}^{(i)}\left(\theta^{(n)}\right)$;
  Error minimization via a gradient descent step
  $\theta^{(n+1)} \leftarrow \theta^{(n)} - \eta\, g$;
  $n \leftarrow n + 1$;
Return $\theta^{(n)}$

Despite the lack of theoretical guarantees of convergence to a global optimum, the simple procedure of random initialization followed by iterative gradient-based updates is sufficient in practice to train complex NN.

1.5 Evaluation

Now that the process of training a model has been described, the question of how to evaluate it must be addressed. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model

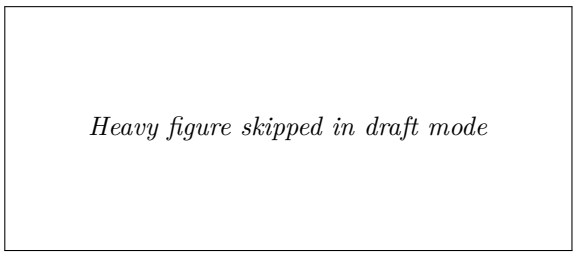


Figure 1.6: *Figure skipped in draft mode (set* `\draftfiguresfalse` *to render it).*

will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, the average loss on the training set, denoted $\bar{\mathcal{L}}_{\text{train}}$, is studied. This is an instance of the performance measure P defined in Equation (1.3), restricted to $\mathcal{D}_{\text{train}}$:

$$\bar{\mathcal{L}}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)},y^{(i)})\in\mathcal{D}_{\text{train}}} \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.18}$$

If $\bar{\mathcal{L}}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what has been discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. Its ability to generalize must now be studied. To do so, the corresponding average loss on $\mathcal{D}_{\text{test}}$ is computed:

$$\bar{\mathcal{L}}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)},y^{(i)})\in\mathcal{D}_{\text{test}}} \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.19}$$

If $\bar{\mathcal{L}}_{\text{test}} \gg \bar{\mathcal{L}}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to

the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\mathcal{L}}_{\text{test}} \approx \bar{\mathcal{L}}_{\text{train}}$, this indicates that the training procedure has successfully produced a NN that generalizes well beyond the training data [Kawaguchi et al., 2022]. The resulting model can therefore be considered suitable for the task for which it was trained, marking the end of the NN training process.

1.6 Conclusion

This chapter has introduced the fundamental concepts underlying DL, including the general objective of NN models, their structure, the training procedure, and their evaluation. For clarity of presentation, several simplifying assumptions have been made throughout this chapter. The problem has been considered in a regression setting, using a MLP trained with SGD and a batch size of one. These choices were made deliberately to expose the core mechanisms of DL as clearly as possible.

Despite these simplifications, the framework introduced here captures mechanisms that are shared across a broad range of DL methods: A sequence of parameter updates, each based on information about the loss around the current parameters. The apparent simplicity of this learning procedure therefore contrasts with the complexity of the functions that NN can represent. Understanding how such complex behavior emerges from a relatively simple optimization process remains an important challenge in the study of DL.

Mathematical tools allow us to describe the functions represented by NN and the procedures used to train them. However, they do not yet fully explain their remarkable empirical performance. Understanding the origin of this success therefore requires looking beyond the optimization procedure itself.

A common feature across NN architectures is the presence of intermediate representations. Whether considering a MLP, a CNN, an LSTM, or an architecture based on attention, the input is progressively transformed into intermediate representations before producing the final output. Their presence provides a common perspective for understanding how NN transform information. The collection of these intermediate representations is referred to as the Latent Space (LS). Studying this LS can provide insights into how NNs learn and generalize, which motivates the following chapter.

Chapter 2

Latent Space Analysis

In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep Learning (DL) introduces a paradigm shift: rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, Neural Network (NN) gradually construct representations that become increasingly relevant to the task under consideration [Rumelhart et al., 1986].

These representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training. Since these representations are learned internally by the network and are not directly observable from the input or output spaces, they are commonly referred to as Latent Representation (LR). The collection of these LR learned throughout the network is referred to as a Latent Space (LS).

These LS are now at the core of a growing number of practical applications. Neural video compression exploits the compactness of LS to encode information efficiently [Lu et al., 2019]. In Reinforcement Learning (RL), agent internal representations are used for efficient exploration in extremely open environments [Burda et al., 2018]. Recommendation systems model users and content within a shared LS to compute meaningful similarities [He et al., 2017]. Retrieval-augmented generation systems index and query knowledge bases through LR [Lewis et al., 2021]. Finally, Large Language Model (LLM) and multimodal architectures rely on the quality of learned representations to align heterogeneous modalities such as text, images, and audio [Radford et al., 2021]. These examples represent only some of the most widespread applications of LR.

Given their extensive use, the study of LR has become a central topic in DL research. A major objective is to understand how information is organized within these spaces, which concepts are encoded, and how these LR influence model behavior.

The remainder of this chapter is organized as follows:

- Section 2.1 introduces the hypothesis that underlies much of the research on LS analysis, which we refer to as the *Concept Hypothesis* (Hyp. 1).
- Sections 2.2, 2.3, 2.4, and 2.5 examine different approaches to LS analysis, respectively covering probing methods, semantic directions, sparse autoencoders, and clustering-based methods.
- Section 2.6 concludes with a comparative synthesis of these methods and a discussion of the open questions that remain.

2.1 Concept Hypothesis

In this section, we introduce the main working hypothesis underlying much of the literature on LS analysis. Although rarely stated explicitly, this hypothesis is implicitly present in the recurring use of notions such as abstraction, representation, LS, concept, and semantics. Throughout this manuscript, we refer to this hypothesis as *Concept Hypothesis* (Hyp. 1). The goal of this section is to make this conceptual framework explicit by defining its main components.

The *Concept Hypothesis* (Hyp. 1) originates from the need to explain the generalization ability of deep NN observed after training, as discussed in Section 1.5. A model is said to generalize when, for an input $x \in \mathcal{X}$ not seen during training, it still produces an output $\hat{y}$ close to the true output y. Such behavior cannot be explained by simple memorization of training samples. Instead, the model must extract and exploit underlying regularities in the target function. This requires a process of abstraction, as defined in Definition 2.1.1, whereby raw inputs are transformed into internal structures that are meaningful for the task [Bengio et al., 2014].

Definition 2.1.1: Abstraction

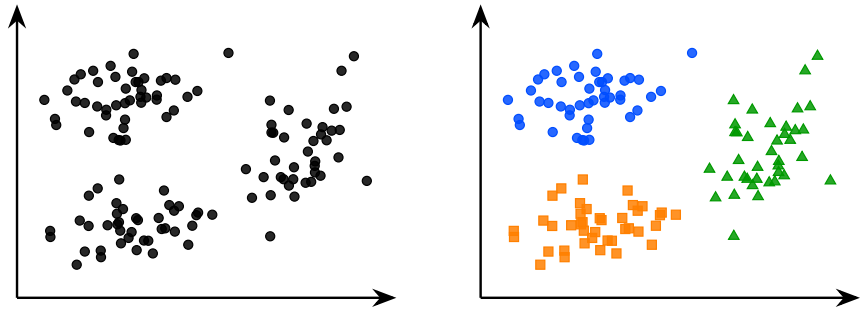
Abstraction is the process by which a model transforms an input into a task-relevant internal form. It captures and restructures information in order to represent relationships that are useful for prediction.

The ability of NN to perform abstraction arises from their intermediate activations. A NN does not directly map x to $\hat{y}$, it computes a sequence of intermediate activations across hidden layers. These activations are numerical encodings of the input referred to as embeddings. Once training is complete, these embeddings are structured to support the performance of a specific task. To mark this distinction, we will refer to these trained embeddings as LR, as defined in Definition 2.1.2.

Definition 2.1.2: Latent Representation

A LR is the internal encoding of an input given by an intermediate layer of a trained NN.

Each hidden layer refines the LR of the previous layer. Each transformation progressively reshapes the LR into a form more directly aligned with the output



(a) Observed activations **(b)** Under the concept hypothesis

Figure 2.1: The same latent space activations seen without **(a)** and under **(b)** the *Concept Hypothesis* (Hyp. 1).

space [Rumelhart et al., 1986]. This process can be described as a sequence of transformations where each $\mathbf{h}_l$ is a LR:

$$x \to \mathbf{h}_1 \to \mathbf{h}_2 \to \cdots \to \mathbf{h}_{L-1} \to \hat{y}$$

The set of all such LR produced by a layer over a dataset defines its LS, as defined in Definition 2.1.3.

Definition 2.1.3: Latent Space

The LS is the set of all LR produced by a layer over a dataset. It is shaped by learning and organizes LR according to task-relevant properties.

Under the *Concept Hypothesis* (Hyp. 1), the analysis of the LS is assumed to provide access to abstract structures learned by the model, as illustrated in Figure 2.1. The same set of LR can be viewed either as an unstructured point cloud or, under this hypothesis, as a collection of points organized by concept membership, in which case the structure of the LS becomes meaningful and amenable to analysis. In Figure 2.1, the colors represent hypothetical concept memberships assigned by the model. In this manuscript, a concept is defined as an abstract representation that organizes information and supports reasoning (see Definition 2.1.4).

Definition 2.1.4: Concept

A concept is a task-dependent abstraction that captures regularities in the input space relevant for predicting the target output.

Our definition of concepts is consistent with both the pragmatic tradition in philosophy and the information-processing view of cognition. Within these frameworks, concepts arise from the need to simplify an otherwise intractable reality. Because the environment contains more information than any cognitive system can process, intelligent behavior relies on abstractions, called concepts, that discard irrelevant details while preserving the information required to achieve a given objective [Peirce, 1878, Misak, 2013]. These concepts are realized as

representations that reduce complexity and enable reasoning under limited computational resources [Newell, 1980, Simon, 1996]. Although these theories were originally developed to explain human cognition, the *Concept Hypothesis* (Hyp. 1) assumes that the same principles can be extended to deep NN.

In order to influence the model's computations, concepts must necessarily admit a concrete realization within the LS. We refer to this realization as the Concept Structure (CS). The definition of the CS is provided in Definition 2.1.5.

Definition 2.1.5: Concept Structure

A CS is the specific form for how a concept is instantiated within the LS.

The methods reviewed in this chapter all build on the *Concept Hypothesis* (Hyp. 1) formalized in Hypothesis 1, but differ in how they characterize the CS of a concept. This choice directly determines how concepts can be identified and extracted from the LS. As a result, several families of LS analysis methods have emerged in the literature, each adopting a different view of how concepts are structured within the LS.

Hypothesis 1: Concept Hypothesis

NN generalize by performing abstraction. Through training, their LS becomes organized around concepts. Concepts are task-dependent abstractions that capture regularities relevant to the task. Analyzing the LS therefore provides access to the CS through which these concepts are instantiated in the model.

Now that the definition of a *Concept Hypothesis* (Hyp. 1) has been established, we can consider its relationship to human. According to our definition, we can imagine concepts that are useful for the machine but do not make sense to humans. This relationship is captured by the notion of semantics [Marconato et al., 2023], defined in Definition 2.1.6. Some concepts developed in LS can be readily associated with human concepts and are said to have high semantic value. Others may contribute to task performance while remaining difficult for humans to understand, and are said to have low semantic value.

Definition 2.1.6: Semantics

Semantics refers to the alignment between concepts in the model LS and concepts used by a human observer. A concept is semantic if it can be associated with a human interpretable concept.

It is important to note that this conceptual framework is not supported by a complete theoretical understanding of deep NN. Current mathematical tools do not provide a rigorous characterization of their internal representations and do not formally prove that concepts emerge as identifiable CS within LS. Nevertheless, this perspective constitutes a widely adopted working hypothesis in the field of representation and LS analysis.

The remainder of this chapter is devoted to reviewing methods for analyzing LS and extracting the concepts they encode. We begin with probing, one of the most direct approaches for assessing whether concepts are represented in the LS.

2.2 Probing

Probing methods aim to determine whether a given semantic concept is encoded within the LS of a NN [Li et al., 2024, Karvonen, 2024]. The idea behind these methods is that if a concept can be reliably decoded from LR, it is likely relevant to the problem being solved. Studying such concepts can therefore provide insight into how the network internally represents the problem.

A probing analysis begins by identifying a set of semantic concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concepts. We denote by c_j the presence or absence of concept j in example x. The dataset can therefore be written as follows:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)}, c_1^{(i)}, \dots, c_n^{(i)})\}_{i=1}^{N}.$$

The data are subsequently projected into the LS of the pretrained model. For an input x, we write $\mathbf{h}_l(x) \in \mathbb{R}^{n_l}$, or simply $\mathbf{h}_l$, the LR obtained at layer l. A classifier, referred to as a probe, is then trained to predict the presence of a given concept from this LR. It is denoted by $p_\theta^{l,j}$, where θ represents its parameters, l indicates the layer from which the LR is extracted, and j identifies the concept being predicted. The probe maps the LR to a scalar value in $[0, 1]$, which can be interpreted as the probability that the concept is present. The parameters θ are learned by minimizing a loss function over the training examples using Stochastic Gradient Descent (SGD). This gives the following mathematical formulation:

$$p_\theta^{l,j} : \mathbb{R}^{n_l} \to [0,1], \qquad \min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(p_\theta^{l,j}\left(\mathbf{h}_l(x^{(i)})\right), c_j^{(i)}\right). \tag{2.1}$$

The architecture of the probe must be kept intentionally simple. A probe with excessive expressive capacity may be able to predict the concept by exploiting complex patterns within the representation. These patterns do not necessarily indicate that the concept is explicitly encoded in the LS. Instead, they may result from the ability of the probe itself to combine and transform the available information. Consequently, a highly expressive probe makes it difficult to distinguish between information that is directly accessible in the representation and information that is reconstructed by the classifier. Restricting the capacity of the probe therefore helps ensure that its predictions primarily reflect the information already present in the LS.

For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the NN and potentially usable during decision-making. If the classifier achieves performance significantly above chance level, this suggests that the LS contains exploitable information related to the considered concept.

Figure 2.2 illustrates the decision boundaries of such linear probes for three concepts, where each colored region indicates the area classified positively by the corresponding probe.

An application of this technique can be found in studies investigating whether LLM develop internal representations of their environment [Li et al., 2024, Karvonen, 2024]. In these works, researchers train LLM architecture to play games such as chess or Othello in a purely textual setting. The model is provided only with sequences of

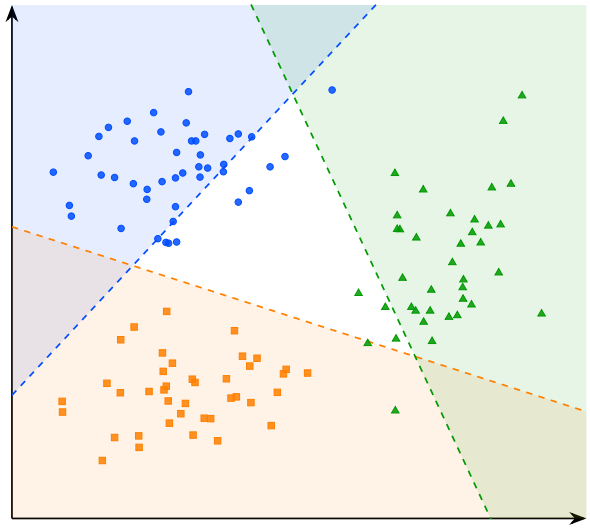


Figure 2.2: Decision boundaries of the linear probes for concepts A, B, and C.

moves and information about the actions it can take, all expressed in textual form. The central question is whether the LLM develops an internal representation of the game state. To address this question, the probed concepts correspond to the presence or absence of different game pieces on specific board positions.

As illustrated in Figure 2.3, probes are able to reconstruct the full board state from the model's LR, even though the only available information to the model is a sequence of text. In this figure, each row corresponds to a piece type, comparing the ground truth with the probe's predictions and confidence. The close agreement between the ground truth and predicted positions suggests that the model develops a structured representation of the underlying environment from which the data are generated.

However, the fact that a concept can be decoded from LR does not necessarily imply that the model actually uses this information to perform its task. To assess whether LR play a causal role in the model's output, it is possible to perform interventions on them. An intervention consists of modifying a LR in a controlled manner by activating or deactivating a given concept, and then studying its impact on the model's output. If the resulting change in the model's output is consistent with the applied perturbation, this suggests that the representation had a causal role.

To this end, the LR is minimally modified such that the probe prediction for the target concept c_j changes state, while the predictions associated with other concepts remain unchanged. This goal can be formulated as an optimization problem over the LR itself. The optimization is performed directly on the LR, using SGD. The solution to this problem is the optimal modified LR, defined as:

$$\mathbf{h}_l^* = \arg\min_{\mathbf{h}'_l} \underbrace{\gamma\, \mathcal{L}_{\text{flip}}\big(p_\theta^{l,j}(\mathbf{h}'_l), c_j\big)}_{\text{flip target concept } c_j} + \underbrace{\lambda \sum_{k \neq j} \mathcal{L}_{\text{preserve}}\big(p_\theta^{l,k}(\mathbf{h}'_l), c_k\big)}_{\text{preserve other concepts } c_{k\neq j}} + \underbrace{\beta \|\mathbf{h}'_l - \mathbf{h}_l\|_2^2}_{\text{minimality}}$$

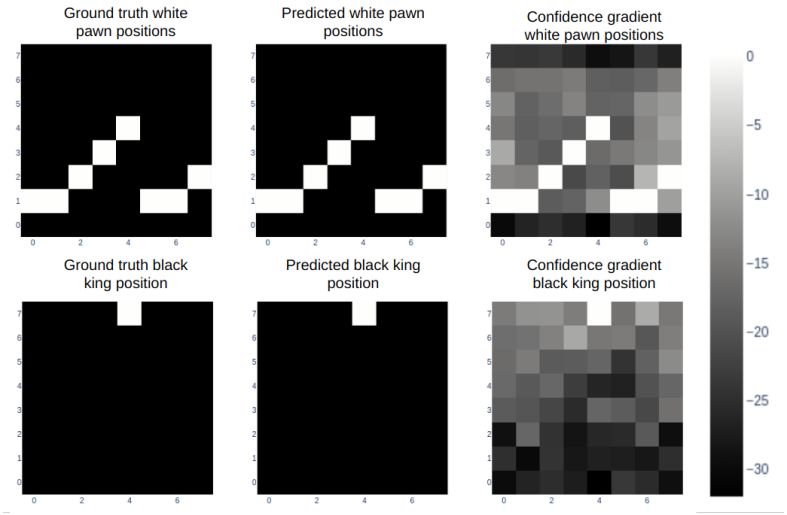


Figure 2.3: Probing analysis of a large language model trained to play chess [Karvonen, 2024].

where:

- $\mathbf{h}_l$ is the original LR of the input at layer l,
- $\mathbf{h}'_l$ is the modified LR being optimized,
- $\mathbf{h}^*_l$ is the optimal modified LR,
- $\mathcal{L}_{\text{flip}}$: loss encouraging the prediction of the target concept c_j to change state (i.e., flipping the predicted label),
- $\mathcal{L}_{\text{preserve}}$: loss enforcing invariance of all non-target concepts c_k for $k \neq j$,
- $\|\mathbf{h}'_l - \mathbf{h}_l\|_2^2$: regularization term ensuring the modified LR remains close to the original one,
- γ, λ, β: weighting coefficients controlling the trade-off between the intervention strength, the preservation of other concepts, and the minimality of the perturbation.

In the context of LLM studied previously, such interventions have been used to remove specific pieces from the model's internal representation of the game state [Li et al., 2024, Karvonen, 2024]. The main observation is that, in the vast majority of cases, the model behaves as if the piece had effectively disappeared. The LLM continues to generate legal moves while correctly accounting for the absence of the removed piece. This provides evidence for the causal role of these internal representations in the model's behavior.

A main limitation of the probing approach is that the concepts under investigation must be predefined before any analysis can be performed. This introduces a strong bias in what can be studied. Moreover, annotating a dataset for each concept is extremely time-consuming and requires significant human effort. Interventions in LS are also difficult to interpret, as it is not always clear

what constitutes a meaningful change in the network's output under a given perturbation.

Moreover, probing methods provide a binary view of concepts, focusing on whether a given concept is present or absent in a LR. This binary perspective can be overly restrictive, as concepts may be represented in a more continuous or graded manner, with varying degrees of presence. To obtain a more detailed characterization of these representations, other approaches have been proposed, such as the analysis of interpretable directions in LS.

2.3 Interpretable Directions

To overcome the limitations of binary concept detection in probing methods, several approaches have been proposed to analyze the structure of LR. Among them, the notion of interpretable directions has emerged as a natural extension, aiming to capture the continuous nature of how concepts are encoded in LS. Empirically, it has been observed that shifting a LR along a direction can induce coherent and semantically meaningful changes in the model's output [Haas et al., 2024, Voynov and Babenko, 2020]. Such transformations can be mathematically express as:

$$\mathbf{h}' = \mathbf{h} + \alpha \mathbf{v}_i,$$

where:

- $\mathbf{h}$ denotes the original LR,
- $\mathbf{v}_i$ is a unit vector associated with concept i,
- α controls the strength of the intervention,
- $\mathbf{h}'$ is the resulting perturbed LR.

In many cases, the resulting change in the model's output varies smoothly with α, suggesting an approximately linear CS with respect to certain semantic factors. This observation has led to the hypothesis that neural representations may organize concepts in a partially linear manner within the LS.

Vectors of this form are referred to as interpretable directions, as they correspond to transformations that can be associated with specific semantic attributes. The objective of this line of work is therefore to identify directions in LS that align with meaningful variations in the encoded concepts. This is illustrated schematically in Figure 2.4, where arrows represent semantic shifts from one concept toward others, and moving along these directions induces a coherent change in the model's behavior, suggesting that the transition between concepts is continuously encoded along these axes.

This phenomenon has been extensively studied in generative image models. As illustrated in Figure 2.5, where each row corresponds to a distinct direction $\mathbf{v}_i$ and each column to an increasing value of α, controlled perturbations of LR along interpretable directions can induce specific semantic transformations, such as stroke thickness modification in digit images, head rotation in facial images, or texture and background changes in natural images [Voynov and Babenko, 2020]. These observations have led to the development of the field of LS editing, whose objective is to manipulate generated content directly through modifications in LS rather than through changes in the input prompt.

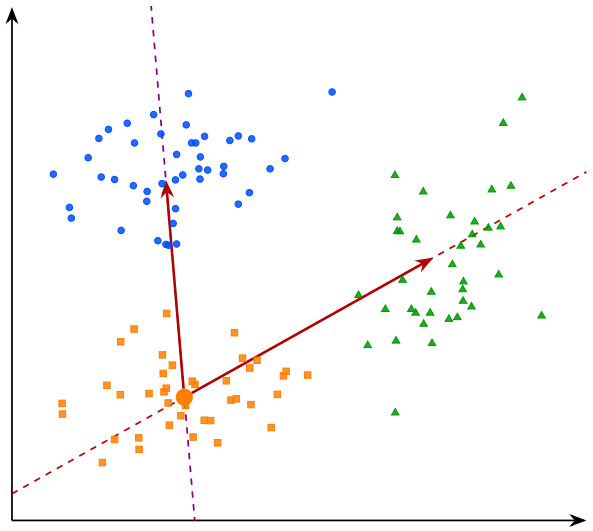


Figure 2.4: Interpretable directions in the latent space.

A central challenge remains identifying such interpretable directions. Most existing approaches rely on auxiliary models that encode human defined semantics, such as CLIP [Haas et al., 2024], to relate latent variations to textual concepts. Consequently, these methods inherit a limitation similar to that of probing techniques, since concept discovery depends on a pre-existing semantic framework defined by humans.

To overcome this limitation, a research direction known as unsupervised semantic discovery has emerged. Its goal is to uncover interpretable directions in LS without explicit semantic supervision. In these approaches, one model applies perturbations along random directions in LS, while a second model attempts to recognize the applied transformation. If a stable correspondence emerges between the perturbation and its prediction, this suggests that the considered direction corresponds to a coherent CS [Voynov and Babenko, 2020]. In this framework, human intervention occurs only a posteriori, to verify whether the discovered directions indeed correspond to interpretable semantic variations.

Despite these advances, several challenges remain. It appears that not all concepts encoded by NN are linearly identifiable in LS. In practice, interpretable direction methods typically discover only a few dozen directions per dataset, which intuitively seems insufficient to account for the full diversity of semantic variations that a generative model is capable of producing.

In addition, LR often exhibit a phenomenon known as entanglement, illustrated in Figure 2.6, where multiple semantic attributes are mixed within the same directions of the LS. For instance, in a generative face model, moving along the “glasse” direction may simultaneously modify age and gender, whereas a disentangled LR would let each axis control a single semantic attribute while leaving the others unchanged. Ideally, one would like each semantic factor to vary independently along a specific direction, without affecting other attributes. This lack of separability indicates that semantic information is not cleanly factored in the LS, which motivates the search for more structured decompositions.

This limitation has motivated a stronger hypothesis about how NN organize

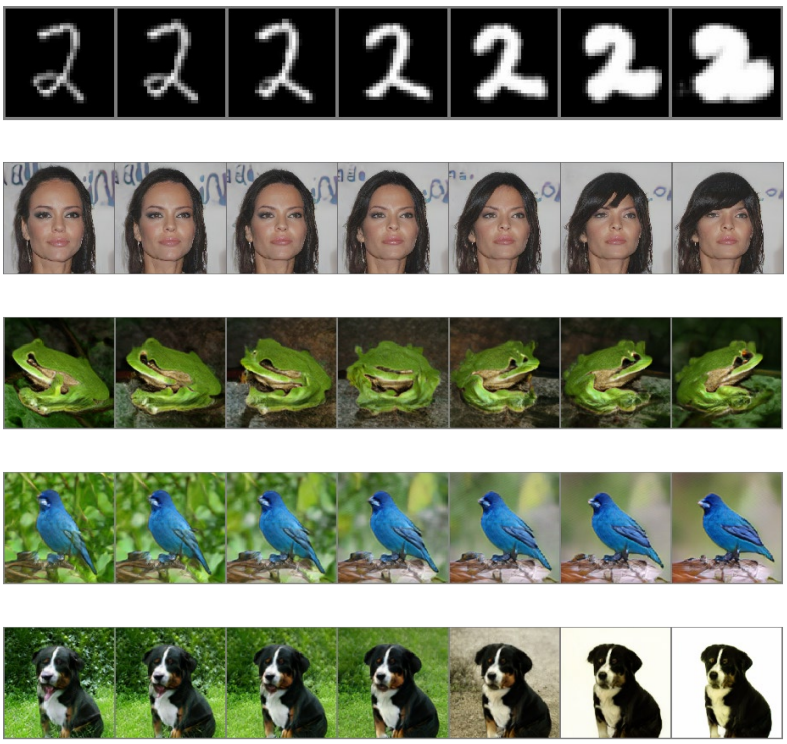


Figure 2.5: Interpretable directions in the latent space of a generative model [Voynov and Babenko, 2020].

information internally, namely the *Superposition Hypothesis* (Hyp. 2), introduced in the next section.

2.4 Sparse Autoencoder

Approaches based on latent directions face limitations. Only a restricted subset of concepts admit a linear representation, and even these directions are often entangled with multiple semantic concepts, as illustrated in Figure 2.6. To address this limitation, an extension of the *Concept Hypothesis* (Hyp. 1) has been proposed under the name *Superposition Hypothesis* (Hyp. 2), formalized in Hypothesis 2 [Elhage et al., 2022].

Hypothesis 2: Superposition Hypothesis

Concepts are preferentially represented by distinct linear directions in LS. When the number of concepts exceeds the available dimensions, multiple concepts are encoded in superposition within the same directions.

According to this hypothesis, in an unconstrained LS, each concept is represented in its own independent linear direction. However, NN operate in finite-dimensional spaces, so when the number of concepts exceeds the available

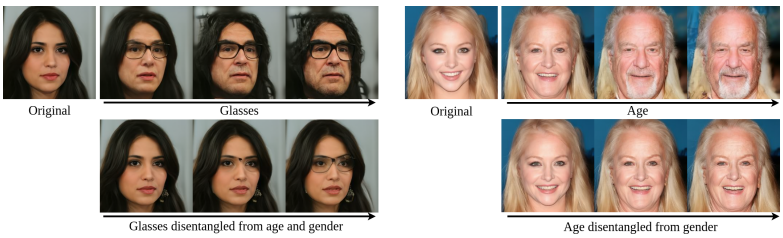


Figure 2.6: Illustration of entanglement in the latent space of a generative face model [Haas et al., 2024].

dimensions, such a one-to-one correspondence becomes impractical. The network must therefore reuse its representational dimensions, allowing multiple concepts to be encoded using the same directions. This shared encoding is referred to as superposition [Bricken et al., 2023]. As a result, in a superposed LR, linear analysis of the LS may not isolate individual concepts, the resulting directions reflect a mixture of several semantic attributes.

The idealized non-superposed LR can be mathematically expressed as follows:

$$\mathbf{h} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i, \qquad \forall i \neq j, \quad \mathbf{v}_i^\top \mathbf{v}_j \approx 0, \qquad \|\boldsymbol{\alpha}\|_0 \ll n,$$

where:

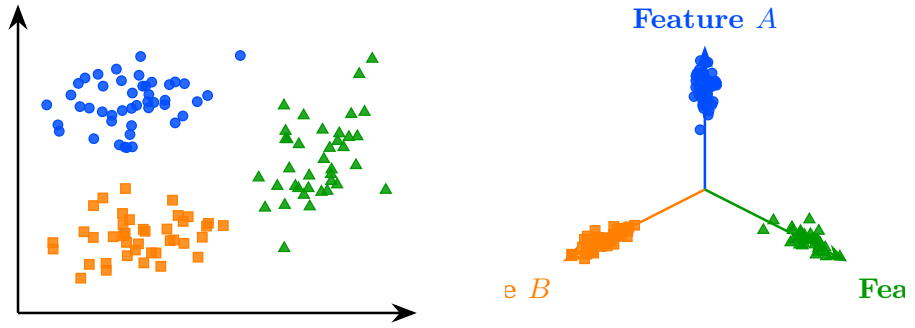
- $\mathbf{h}$ denotes a LR,
- n is the total number of concepts,
- $\mathbf{v}_i$ represents the direction associated with concept c_i,
- α_i corresponds to the activation strength of concept c_i in the LR $\mathbf{h}$,
- $\mathbf{v}_i^\top \mathbf{v}_j \approx 0$ indicates that concept directions are approximately orthogonal, meaning each concept independently controls a single factor of variation,
- $\|\boldsymbol{\alpha}\|_0 \ll n$ indicates that only a small fraction of concepts are active for a given LR.

Under the *Superposition Hypothesis* (Hyp. 2), the phenomenon of superposition becomes an obstacle for any method that attempts to analyze LR directly in the original activation space. This has motivated the development of methods that aim to recover a less superposed LR. A common approach is to map LS into a higher-dimensional space while imposing additional constraints that encourage individual concepts to be represented separately. This transformation is illustrated in Figure 2.7, while LR are entangled in the initial LS, the sparse representation aligns each concept with a dedicated feature axis.

SAE provide a practical implementation of this approach and can be defined as follows:

$$z = f_{\text{enc}}(\mathbf{h}), \quad \hat{\mathbf{h}} = f_{\text{dec}}(z)$$

$$\mathcal{L}_{\text{auto}} = \gamma \|\mathbf{h} - \hat{\mathbf{h}}\|_2^2 + \lambda \|z\|_1$$



(a) Dense latent space **(b)** Sparse SAE space

Figure 2.7: Transformation from a dense latent space **(a)** to a sparse Sparse Autoencoder (SAE) representation **(b)**.

where:

- $f_{\text{enc}}(\mathbf{h})$ is the encoding function that projects the original LR $\mathbf{h}$ into a higher-dimensional sparse space,
- $f_{\text{dec}}(z)$ is the decoding function that reconstructs the original LR from the sparse code,
- $z \in \mathcal{Z}$ is the sparse representation of $\mathbf{h}$, with $\dim(z) \gg \dim(\mathbf{h})$,
- $\hat{\mathbf{h}}$ is the reconstructed LR,
- $\gamma\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2$ is the reconstruction loss ensuring that the original LR can be faithfully recovered,
- $\lambda\|z\|_1$ is the sparsity regularization term that encourages most components of z to be zero,
- γ, λ are weighting coefficients controlling the trade-off between reconstruction fidelity and sparsity.

In practice, both f_{enc} and f_{dec} are implemented as linear projections between the original LS and the high-dimensional space $\mathcal{Z}$. This design follows the same principle as in probing methods, the transformation used to analyze the LR should remain as simple as possible. More expressive projection functions could themselves introduce complex structures into the transformed LR. In that case, it would become difficult to determine whether the observed concept reflects a CS already present in the model's or structure introduced by the projection.

The parameters of these projection functions f_{enc} and f_{dec} are learned by minimizing $\mathcal{L}_{\text{auto}}$ using SGD. Under this formulation, each dimension of $\mathcal{Z}$ is interpreted as a dedicated concept direction.

The method proves to be powerful when successfully applied to language models [Templeton et al., 2024]. In these applications, SAE are used to desuperpose the LR of LLM. Once this LR is obtained, it has been shown that many concept vectors exhibit semantic properties. A example is the Golden Gate Bridge concept direction, illustrated in Figure 2.8, which activates systematically

Figure 2.8: A concept direction corresponding to the Golden Gate Bridge, discovered by a sparse autoencoder applied to a large language model [Templeton et al., 2024].

whenever the input text or image refers to the Golden Gate Bridge, regardless of the language or modality. This demonstrates that the learned concept is not tied to a specific surface form but captures a deeper semantic abstraction shared across modalities.

SAE also provide a way to intervene directly on the LR learned by the model. A concept can be artificially activated by modifying its corresponding feature in the sparse representation z, even when the input contains no reference to it. The modified LR is then projected back into the original activation space, after which the model proceeds with standard next-token generation from $\mathbf{h}^*$:

$$\mathbf{h}^* = f_{\text{dec}}(z^*), \quad \text{where} \quad z^* = z + \delta\, e_i,$$

where:

- z is the original sparse representation of the input,
- z^* is the modified sparse representation,
- e_i is the basis vector of $\mathcal{Z}$ associated with the target concept direction,
- δ controls the strength of the artificial activation,
- $\mathbf{h}^*$ is the resulting edited LR in the original activation space.

With this method, a reference to the Golden Gate Bridge can be injected into the LLM's internal representation. This causes the model's output to shift toward discussing the Golden Gate Bridge, even when the prompt contains no reference to it. This provides evidence that the learned feature plays a causal role in the model's behavior. Such interventions enable control over specific model behaviors through LS editing.

Despite their empirical success, *Superposition Hypothesis* (Hyp. 2) exhibit several important limitations that question their assumptions. These limitations can be summarized as follows:

- **Instability of the decomposition.** Repeating the SAE training process multiple times on the same model does not yield the same decomposition of the LS. Different runs produce different concept directions in $\mathcal{Z}$, even when achieving comparable reconstruction performance, which raises questions about the reliability of the discovered CS.
- **Lack of semantic coverage.** In the high-dimensional space $\mathcal{Z}$, a large fraction of dimensions do not correspond to any interpretable semantic concept. This abundance of non-semantic directions raises questions about whether the decomposition truly reflects the model's CS.
- **Sparsity–semantics trade-off.** Increasing the sparsity coefficient λ beyond a certain threshold degrades the semantic quality of the learned features [Jermyn et al., 2024]. Suggesting that enforcing excessive sparsity comes at the cost of interpretability.
- **Failure of scaling to remove superposition.** Under the *Superposition Hypothesis* (Hyp. 2), increasing the dimensionality of the LS should reduce superposition. However, this is not observed in practice. Even in very high-dimensional LS, representations remain superposed, indicating that scaling alone does not resolve the entanglement.

These limitations suggest that *Superposition Hypothesis* (Hyp. 2), while powerful, do not fully resolve the problem of CS. Convincing empirical results do not necessarily validate the underlying theoretical assumptions.

Probing, interpretable directions, and *Superposition Hypothesis* (Hyp. 2) all rest on a shared assumption that the CS can be recovered through linear decompositions of LR. This assumption has driven significant empirical progress, yet it remains theoretically fragile. The following section therefore introduces a approach that relaxes this constraint.

2.5 Clustering

All the LS analysis methods presented so far rely on assumptions about a linear CS. For probing methods, concepts must be at least partially linearly separable for simple probes. Latent directions assumes that variations associated with a concept can be captured by moving along a linear axis of the LS. The *Superposition Hypothesis* (Hyp. 2) assumes that concepts are represented through linear directions that become entangled because of representational capacity constraints. Although these assumptions have led to empirical successes, several observations suggest that the linearity hypothesis may not fully capture the CS.

For instance, some concepts can only be recovered using non-linear probes. Likewise, methods based on interpretable directions typically identify only a limited number of semantic factors. Finally, in sparse autoencoder approaches, enforcing excessive sparsity often degrades reconstruction quality and interpretability, suggesting that the underlying representation may not be perfectly described by a sparse linear decomposition. Taken together, these observations

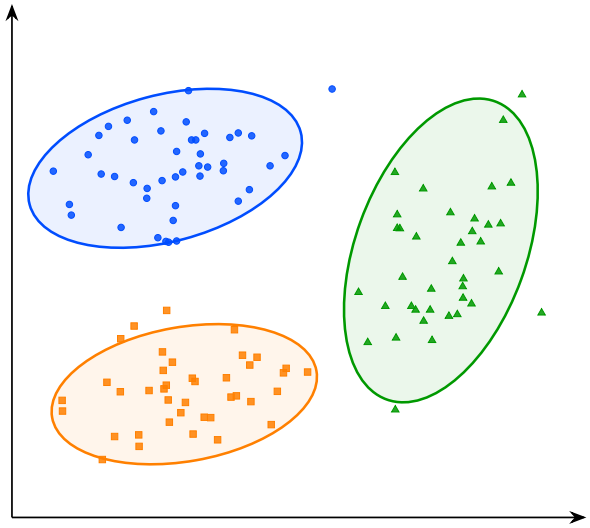


Figure 2.9: Visualization of the latent space structure.

indicate that semantic concepts may not always exhibit a linear CS. This motivates the exploration of alternative approaches that rely on weaker assumptions regarding the geometry of LS.

To the best of our knowledge, clustering-based analysis is the only family of LS analysis methods that does not rely on any assumption of a linear CS [Ren et al., , Wei et al.,]. Rather than assuming that concepts correspond to directions, clustering methods only assume that conceptual similar inputs produce nearby LR. Conversely, inputs that differ significantly from a concept perspective are expected to be located farther apart in the LS. This is a considerably weaker assumption, in the sense that it is empirically verified in a wide range of settings.

Under this assumption, the discovery of concepts can be reformulated as the identification of groups of LR that occupy the same region of the LS. This is shown schematically in Figure 2.9, where the points represent LR associated with concepts A, B, and C, and the ellipses indicate the approximate extent of each group and highlight their geometric separation.

This idea forms the basis of the research field known as deep clustering. In these approaches, a collection of inputs is projected into a LS using a NN. The resulting LR form a point cloud on which standard clustering algorithms can be applied. The obtained clusters are then interpreted as representing distinct concepts.

Deep clustering has been particularly successful in unsupervised image classification. For this type of task, a specific NN architecture called an autoencoder is commonly used. An autoencoder is composed of an encoder f_{enc} and a decoder f_{dec}. The encoder maps an input $x \in \mathcal{X}$ to a low-dimensional LR $\mathbf{z} \in \mathbb{R}^d$, the LR produced by the encoder. The decoder then maps this LR back to the input space to produce a reconstruction $\hat{x}$. The model is trained by SGD to minimize the reconstruction error between the original input and its reconstruction using a Mean Squared Error (MSE) loss. This objective encourages the LS to preserve the most informative features of the input while enforcing a compact representation.

Figure 2.10: Clusters discovered by a deep clustering model applied to natural images [Xie et al.,].

This can be expressed mathematically as follows:

$$x \xrightarrow{f_{\text{enc}}} \mathbf{z} \xrightarrow{f_{\text{dec}}} \hat{x}, \qquad d \ll \dim(\mathcal{X}), \qquad \mathcal{L}_{\text{rec}} = \frac{1}{N}\sum_{i=1}^{N} \left\| x^{(i)} - \hat{x}^{(i)} \right\|_2^2.$$

Once the autoencoder has been trained sufficiently well, the reconstruction loss $\mathcal{L}_{\text{rec}}$ should be low enough for the learned representation to be useful.

A clustering algorithm can then be applied to the compressed representations of the images. Let $\mathcal{D}$ denote a dataset of inputs, through the encoder f_{enc}, this dataset is mapped to a point cloud in the LS:

$$\mathcal{Z} = \{\mathbf{z}^{(i)} \in \mathbb{R}^d \mid \mathbf{z}^{(i)} = f_{\text{enc}}(x^{(i)}),\ x^{(i)} \in \mathcal{D}\}.$$

A classical clustering algorithm is then applied to the set $\mathcal{Z}$ of LR to identify groups of similar LR. Each resulting cluster can therefore be interpreted as a group of images that the model considers similar.

An example of this methodology can be observed in Figure 2.10, where the approach is applied to the STL-10 dataset [Coates et al., 2011] using k-means clustering [MacQueen, 1967] with a number of clusters arbitrarily fixed to $K = 10$ [Xie et al.,]. For each cluster, the corresponding line shows the 10 samples closest to the associated centroid in the LS.

We observe that the method captures coherent semantic categories such as animals, vehicles, and flying entities. This result is particularly striking given that the only available information to the system consists of raw images. It suggests that the compression process induces a LS in which semantically meaningful CS emerge.

More advanced approaches combine representation learning and clustering within a single optimization process. In this setting, additional loss terms encourage LR to organize themselves around cluster centroids during training. This often produces LS that are easier to analyze and improves the quality of the discovered semantic CS [Guo et al., , Miklautz et al.,].

Despite its effectiveness, clustering-based analysis presents several important limitations. First, as with most unsupervised semantic discovery techniques, interpreting the resulting clusters remains partly subjective. Benchmark datasets often provide class labels for evaluation, but the discovered clusters may not match the categories that humans consider relevant.

Alternative semantic groupings may also be equally valid. Second, clustering methods are highly sensitive to design choices, as the number of clusters, the distance metric, and the clustering algorithm itself can all significantly influence the results, making parameter selection challenging and often requiring substantial empirical tuning.

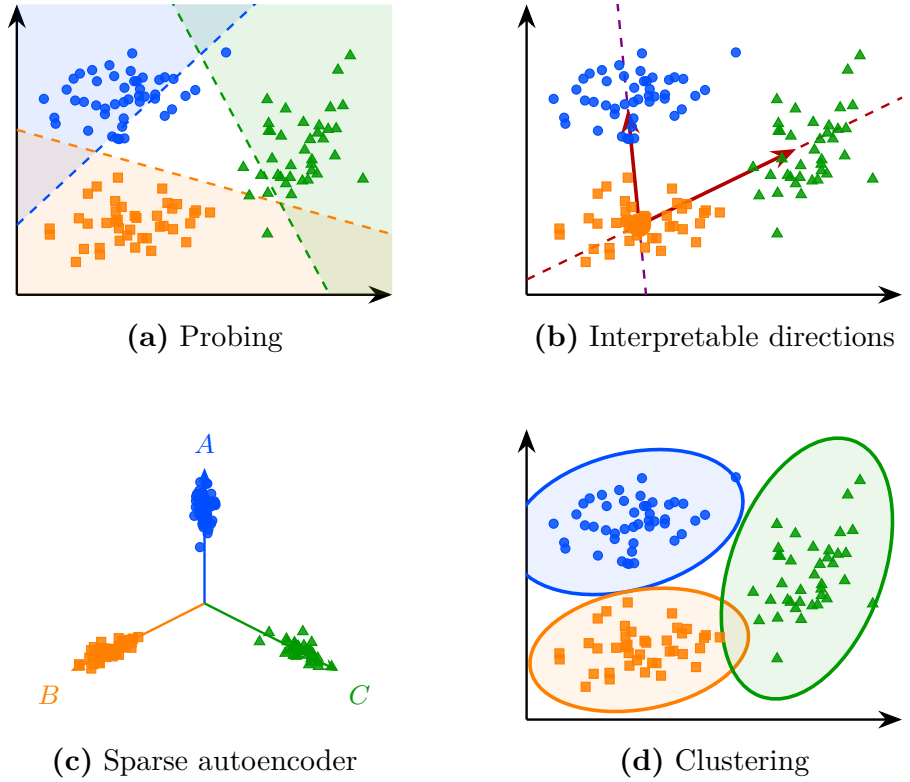
Finally, to the best of our knowledge, no intervention technique has yet been proposed in the context of clustering-based analysis. Unlike probing or SAE analysis, which allow targeted modifications of LR to study their causal role, clustering methods currently offer no mechanism to alter the model's behavior by acting directly on the discovered CS.

2.6 Conclusion

The four families of methods reviewed in this chapter are summarized in Figure 2.11. These include probing, interpretable directions, sparse autoencoders, and clustering, all of which build upon the *Concept Hypothesis* (Hyp. 1) introduced in Section 2.1. Each of them approaches the question from a different angle, but they share a common objective: uncover how LR are structured and what information can be extracted from them.

However, a fundamental difficulty remains. There is currently no consensus on how to rigorously evaluate whether these methods truly capture the internal concepts of a model. Most existing metrics assess reconstruction quality or linear separability, but these do not directly measure whether the discovered CS correspond to meaningful abstractions from the model's perspective. This leaves open the question of what it means, formally, for a concept to be represented in a NN.

Addressing this question may require contributions from disciplines beyond computer science and mathematics. As the notion of semantics is inherently tied to human interpretation, a purely formal or computational account may be insufficient. Insights from philosophy, cognitive science, or psychology could prove valuable in grounding the notion of concept in a way that is both theoretically rigorous and empirically tractable. Despite these open questions, the methods studied in this chapter provide a practical foundation for analyzing the internal representations of NN.



(a) Probing **(b)** Interpretable directions

(c) Sparse autoencoder **(d)** Clustering

Figure 2.11: Overview of the four latent space analysis methods studied in this chapter, applied to the same point cloud.

Building on the *Concept Hypothesis* (Hyp. 1) and the insights developed throughout this chapter, this manuscript investigates a representation-based approach to explainability in RL. To establish the necessary foundations, the next chapter introduces the principles of RL. The subsequent chapter then presents the proposed method for analyzing these representations and studying their potential for explainability in RL (Chapter **??**).

Acronyms

CNN Convolutional Neural Network 2, 15
CS Concept Structure 19, 23, 24, 27, 29, 30, 32

DL Deep Learning 1–7, 9, 14–16

LLM Large Language Model 3, 16, 20–22, 27, 28
LR Latent Representation 16–18, 20–24, 26–32
LS Latent Space 15–20, 22–32
LSTM Long Short-Term Memory 2, 15

MLP MultiLayer Perceptron 5–7, 15
MSE Mean Squared Error 4, 30

NN Neural Network 1–13, 15–17, 19, 20, 24, 25, 30, 32

RL Reinforcement Learning 16, 33

SAE Sparse Autoencoder 26–29, 32
SGD Stochastic Gradient Descent 10, 12, 15, 20, 21, 27, 30

Compiled on 2/9/2026 at 14:28

Bibliography

[Bengio et al., 2014] Bengio, Y., Courville, A., and Vincent, P. (2014). Representation learning: A review and new perspectives.

[Bricken et al., 2023] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread. **Web**.

[Brown et al., 2020] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

[Burda et al., 2018] Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., and Efros, A. A. (2018). Large-scale study of curiosity-driven learning.

[Coates et al., 2011] Coates, A., Ng, A. Y., and Lee, H. (2011). An analysis of single-layer networks in unsupervised feature learning. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS), pages 215–223.

[Covington et al., 2016] Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198.

[Elhage et al., 2022] Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., et al. (2022). Toy models of superposition. Transformer Circuits Thread. **Web**.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). Deep learning, volume 1. MIT press Cambridge.

[Guo et al.,] Guo, X., Gao, L., Liu, X., and Yin, J. Improved deep embedded clustering with local structure preservation. In Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, pages 1753–1759. International Joint Conferences on Artificial Intelligence Organization.

[Haas et al., 2024] Haas, R., Huberman-Spiegelglas, I., Mulayoff, R., Graßhof, S., Brandt, S. S., and Michaeli, T. (2024). Discovering interpretable directions in the semantic latent space of diffusion models. In 2024 IEEE 18th International Conference on Automatic Face and Gesture Recognition (FG), pages 1–9. IEEE.

Compiled on 2/9/2026 at 14:28

[He et al., 2017] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering.

[Hilbert and López, 2011] Hilbert, M. and López, P. (2011). The world's technological capacity to store, communicate, and compute information. science, 332(6025):60–65.

[Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8):1735–1780.

[Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. Neural networks, 2(5):359–366.

[Jermyn et al., 2024] Jermyn, A., Templeton, A., Batson, J., and Bricken, T. (2024). Tanh penalty in dictionary learning. **Web**.

[Jumper et al., 2021] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589.

[Karvonen, 2024] Karvonen, A. (2024). Emergent world models and latent variable estimation in chess-playing language models.

[Kawaguchi et al., 2022] Kawaguchi, K., Bengio, Y., and Kaelbling, L. (2022). Generalization in Deep Learning, page 112–148. Cambridge University Press.

[Krizhevsky et al., 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.

[LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural computation, 1(4):541–551.

[Lewis et al., 2021] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive nlp tasks.

[Li et al., 2024] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. (2024). Emergent world representations: Exploring a sequence model trained on a synthetic task.

[Lu et al., 2019] Lu, G., Ouyang, W., Xu, D., Zhang, X., Cai, C., and Gao, Z. (2019). Dvc: An end-to-end deep video compression framework.

[MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings ofthe 5th Berkeley Symposium on Mathematical Statisticsand Probability, volume 1, pages 281–297. **Article**.

[Marconato et al., 2023] Marconato, E., Passerini, A., and Teso, S. (2023). Interpretability is in the mind of the beholder: A causal framework for human-interpretable representation learning.

[McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133.

[Miklautz et al.,] Miklautz, L., Klein, T., Sidak, K., Leiber, C., Lang, T., Shkabrii, A., Tschiatschek, S., and Plant, C. Breaking the reclustering barrier in centroid-based deep clustering.

[Minsky and Papert, 1969] Minsky, M. and Papert, S. (1969). An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104.

[Misak, 2013] Misak, C. (2013). The American Pragmatists. Oxford University Press.

[Mitchell, 1997] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York.

[Newell, 1980] Newell, A. (1980). Physical symbol systems. Cognitive Science, 4(2):135–183.

[Peirce, 1878] Peirce, C. S. (1878). How to make our ideas clear. Popular Science Monthly, 12:286–302.

[Radford et al., 2021] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision.

[Raina et al., 2009] Raina, R., Madhavan, A., Ng, A. Y., et al. (2009). Large-scale deep unsupervised learning using graphics processors. In Icml, volume 9, pages 873–880.

[Ren et al.,] Ren, Y., Pu, J., Yang, Z., Xu, J., Li, G., Pu, X., Yu, P. S., and He, L. Deep clustering: A comprehensive survey.

[Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6):386.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.

[Simon, 1996] Simon, H. A. (1996). The Sciences of the Artificial. MIT Press, 3 edition.

[Telgarsky, 2016] Telgarsky, M. (2016). Benefits of depth in neural networks. In Conference on learning theory, pages 1517–1539. PMLR.

[Templeton et al., 2024] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., et al. (2024). Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. Transformer Circuits Thread. **Web**.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.

[Voynov and Babenko, 2020] Voynov, A. and Babenko, A. (2020). Unsupervised discovery of interpretable directions in the gan latent space.

[Wei et al.,] Wei, X., Zhang, Z., Huang, H., and Zhou, Y. An overview on deep clustering. 590:127761.

[Xie et al.,] Xie, J., Girshick, R., and Farhadi, A. Unsupervised deep embedding for clustering analysis.